PYTHON PROJECT FIELD GUIDE · 2026.10

从 dsh-bili-scroll 到懂球帝数据项目

Python 初学者的项目对照与扩展指南

> **保留插件编排边界，把平台数据能力落到可独立运行的 Python CLI。**

代码结构导读　　面向 Python 初学者与前端开发者　　2026-10-25

已实现、离线支持与待核验的接口分开说明

01 / FIELD GUIDE

摘要

一句话结论

两个项目都使用 DSH 插件外壳，但 Python 的交接方式不同：`dsh-bili-scroll` 通过 MCP stdio 提供平台工具，`dsh-dqd-scroll` v0.3.0 通过固定参数启动自编 Python CLI。给懂球帝加功能时，先沿“数据来源 → 校验与加工 → 存储或报告 → CLI → 插件”推进；命令能独立运行后，再接设置页或调度。

当前项目状态

> **先从无网络的赛程日报读懂 Python 调用链，再看动态日报的 HTTP 请求。** dsh-dqd-scroll v0.3.0 的动态日报已支持 `teams.json` 中的 20 支英超球队在线读取和定时运行；球队名单、赛程仍通过本地样本解析和导入，球员资料与统计接口尚未接入。把这几类状态分开，能避免将“某类数据能在线读”误认为“所有数据接口都已实现”。

当前懂球帝项目可以分成两条 Python 数据路径：

1. **本地 SQLite 路径**：从样本导入球队名单与赛程，再由 `schedule-report` 或 `daily-report` 读取数据库生成 Markdown。报告生成不发网络请求。
2. **动态在线路径**：Python 用标准库请求已核验的球队动态只读接口，按兴趣词筛选，写出两份 Markdown；DSH 插件按设定时间启动这条 CLI 命令。动态正文不会写入 SQLite。

这份指南以 2026-10-25 工作区快照为依据：`dsh-bili-scroll` 的插件源码和 `package.json`，以及 `dsh-dqd-scroll` v0.3.0 的源码、README、Python 包配置和 `docs/api-contract.md`。动态接口的 GET、HTTP 200、业务 `code=0` 与下一页重放记录来自 DQD 契约文档；本次没有另行发送网络请求，也没有运行两个项目的业务命令。名单、赛程和球员接口是否能在线调用，仍需重新核实请求证据。

目录

摘要 2

第一幕｜两个项目共用宿主，Python 接法不同 4

第二幕｜从一条 Python 命令读懂 dsh-dqd-scroll 8

第三幕｜沿着现有结构增加一项功能 11

工具箱｜后续迭代时保留哪些判断 14

参考资料 15

第一幕｜两个项目共用宿主，Python 接法不同

1. 先看数据怎样经过两个项目

`dsh-bili-scroll` 的 Node 插件负责 DSH 设置、任务编排和 MCP 客户端；内置的 Python MCP 服务提供 B 站平台工具。两端通过标准输入输出（stdio）交换 MCP / JSON-RPC 消息。



`dsh-dqd-scroll` 保留 Node 插件作为 DSH 外壳，但把 Python 接口改成 CLI。插件用 `spawn` 执行固定的 Python 命令；Python 包通过 `argparse` 选择子命令，不启动 MCP 服务。



图中的 Python 子模块不代表每条命令都会经过所有模块。动态命令经过 `api/` 和报告 formatter，但不经过 `repository.py`；赛程日报经过 `repository.py` 与 `lib/report.py`，不访问网络。先识别每条命令真正经过的路径，再读对应文件。

2. 两个项目的关键差异

比较点	dsh-bili-scroll	dsh-dqd-scroll	对后续开发的启发
发布版本	npm 包 `2.1.0`	npm 包与 Python 包元数据均为 `0.3.0`	版本号是各自项目的发布标识，不代表功能可以直接互换
DSH 外壳	Node.js 插件提供设置、工具注册和任务编排	Node.js 插件提供设置 API、用户数据迁移、环境准备和定时任务	宿主相关职责放在 JS 层，Python 数据规则保持可独立调用
Python 的入口	MCP server；Node 端通过 `lib/mcp.mjs` 通信	`python -m dqd_scroll <命令>`；入口是 `__main__.py` 和 `cli.py`	DQD 新能力先做成明确的 CLI 子命令，无需增加 MCP
平台适配	项目内置 `vendor/bilibili-mcp`，以 MCP 工具提供能力	`src/dqd_scroll/api/` 中维护懂球帝请求，目前在线实现限于球队动态	B 站地址、字段和工具不能直接移植到懂球帝
一轮任务	DSH 可以启动带任务章程的 headless 会话，由模型选择已注册工具处理内容	命令模式直接运行 Python；模型模式也要求 headless DSH 启动同一条 Python 命令	当前 DQD 的筛选和报告规则仍在 Python；模型模式不等于把业务逻辑迁入模型

比较点	dsh-bili-scroll	dsh-dqd-scroll	对后续开发的启发
本机数据	插件用户目录存放配置、凭证、状态和报告	插件模式下，DSH 用户目录保存配置、Python 环境和调度状态；数据库路径来自配置；日报写入相应配置目录或工作目录下的 `out/`	配置、数据库、报告与安装代码各自有明确位置
Python 环境	Bili 的 Python MCP 按上游项目要求安装依赖	要求 Python 3.11；插件可用 uv 在 DSH 用户目录准备环境；Python 包依赖为 `tzdata`	新依赖先确认标准库是否足够，再更新 `pyproject.toml` 和环境安装流程
当前外部动作	具备多种平台工具，部分功能可执行评论等操作	动态任务是只读 GET；名单和赛程仅有离线导入；没有评论或平台写入命令	每个新命令都要标明会不会访问网络、改数据库或产生平台副作用

可复用的是职责划分、受限入口、用户数据与安装包分离等设计；不可复用的是平台 API、字段、认证方式和 Python 工具实现。MCP 是一种调用协议，不是“所有 Python 后端都必须使用”的框架。

当前源码还存在一个版本显示不一致：`package.json` 与 `pyproject.toml` 都声明 `0.3.0`，但 Python 的 `--version` 参数仍输出 `0.1.1`，`src/dqd_scroll/__init__.py` 的 `__version__` 是 `0.1.0`。报告按 npm 与 Python 包元数据称当前项目为 v0.3.0；后续发版前应把 CLI 和模块版本显示同步。仓库已有发布元数据检查会比较 npm 与 Python 包版本，但该检查不覆盖这两个 Python 版本字符串。

v0.3.0 的插件层调用链

v0.3.0 不只是多了一条动态 API：插件设置、Python 安装、数据库初始化、定时运行和报告日志已经连成可用的宿主流程。读懂这条链，能看出哪些工作属于 JavaScript 插件，哪些属于 Python。

阶段	主要文件	Python 初学者要理解的事
插件启动	`index.js::apply()`、`lib/user-data.js::initializeUserData()`	DSH 提供用户目录；插件把配置、数据库和旧日报迁入该目录，避免写进安装目录
设置读写	`client.js`、`settings-api.js`	页面通过本机 API 保存球队、兴趣词、时间与执行方式；JS 校验表单并更新 TOML
环境准备	`settings-api.js`、`lib/uvfetch.mjs`	插件下载并校验 uv，再准备 Python 3.11、安装 `tzdata` 并调用 `db-init`
自动调度	`lib/scheduler.js`、`lib/dynamics-scheduled-runner.js`	Windows 注册系统任务；macOS 由插件内每 30 秒检查一次上海时间
Python 工作	`__main__.py` → `cli.py` → `workflow.py` → `api/`、`lib/`	CLI 解析输入；业务层请求、检查、筛选并生成 Markdown
运行记录	`lib/logger.js`、`dynamics-schedule-state.json`	日志和最近执行状态保存在 DSH 用户数据目录，设置页能查看

在命令执行模式下，调度器将固定的 `dynamics-report` 参数直接交给 Python。模型执行模式会启动 headless DSH，并要求它原样运行同一条 Python 命令、检查结果文件；数据抓取、筛选和 Markdown 格式仍由 Python 代码负责。设置页的“现在跑一次”使用用户选定的起止时间，并将手动运行状态单独记录。

3. 懂球帝功能要按证据分成三类

功能	当前实现	证据边界
球队动态日报	Python 在线请求 GET，解析 `data.articles` 和 `data.next`，筛选兴趣词，生成全部动态与兴趣命中两份报告；DSH 可按配置时间调度	`docs/api-contract.md` 记载用户在 mitmweb 观察并在 Postman 重放首屏和下一页，HTTP 200、业务 `code=0`。程序不保存抓包 Cookie、Token 或设备标识；动态正文只在本次进程处理，不写 SQLite
球队名单	从本地 JSON 响应样本解析、校验并替换 SQLite 中的人员关系	没有在线名单请求；历史请求线索不能证明当前认证、状态码或请求参数
球队赛程	从本地 JSON 样本解析当前赛季、核对 URL、去重并写入 SQLite；`schedule-report` 和 `daily-report` 只读数据库	没有在线赛程请求；响应样本中的赛季 URL 不是实际 HTTP 请求记录
球员资料、统计与能力值	暂无对应在线数据流程	`docs/api-contract.md` 中的 Postman URL 是待复核线索，字段、参数和认证要求仍需核验

Python 项目目前初始化五张 SQLite 表：`teams`、`players`、`team_players`、`league_seasons` 和 `team_schedules`。动态文章不在这些表里。`status` 命令也明确说明了动态只读日报已经接入，而名单、赛程和球员接口的实时 HTTP 契约仍待核验。

4. 对照源码时先抓住每边的“交接点”

先从两个入口文件各追一条路径：B 站是 `index.js` → `lib/mcp.mjs` → `vendor/bilibili-mcp/mcp_server.py`；懂球帝是 `index.js` → `src/dqd_scroll/__main__.py` → `cli.py`。到 Python 端以后，再按本章后面的文件职责表继续读 `api/`、`lib/`、`workflow.py` 和 `repository.py`。

不要从大文件第一行读到最后一行。先追踪一个函数调用，再沿着实际被调用的模块深入；这样读到的是一条具体的数据流，而不是一堆互不相连的文件名。

第二幕｜从一条 Python 命令读懂 dsh-dqd-scroll

5. 从没有网络的赛程日报开始

第一条适合初学者追踪的命令是 `daily-report`：数据已经在 SQLite 中，生成报告不需要请求懂球帝，也不更新数据库。它能把 Python 的入口、命令分发、数据库读取、格式化和文件输出串成一条线。

从项目根目录运行的示例：

```
python -m dqd_scroll daily-report --database data/dqd_scroll.sqlite3 --team-id 50000529 --as-of 2026-10-25
```

`--as-of` 固定报告日期，便于重复生成同一日历月的报告。代码调用顺序如下：

daily-report 的完整本地调用链



顺序	文件或函数	初学者可以问自己什么?
1	`__main__.py`	用户运行 `python -m` 时，哪一个 Python 文件先执行?
2	`cli.py::build_parser()`	命令名和参数如何变成 `args`? 哪些参数是必填?
3	`cli.py::main()`	`args.command` 怎样选中具体处理函数?

顺序	文件或函数	初学者可以问自己什么?
4	`generate_daily_report()`	它拿到什么参数，调用哪些读取和格式化函数?
5	`Repository.get_team_schedule()`	SQL 从哪张表读数据？赛季怎样被限定?
6	`format_daily_schedule_report()`	输入数据怎样变成表格文字?
7	`Path.write_text()`	输出文件在哪个目录，编码是什么?

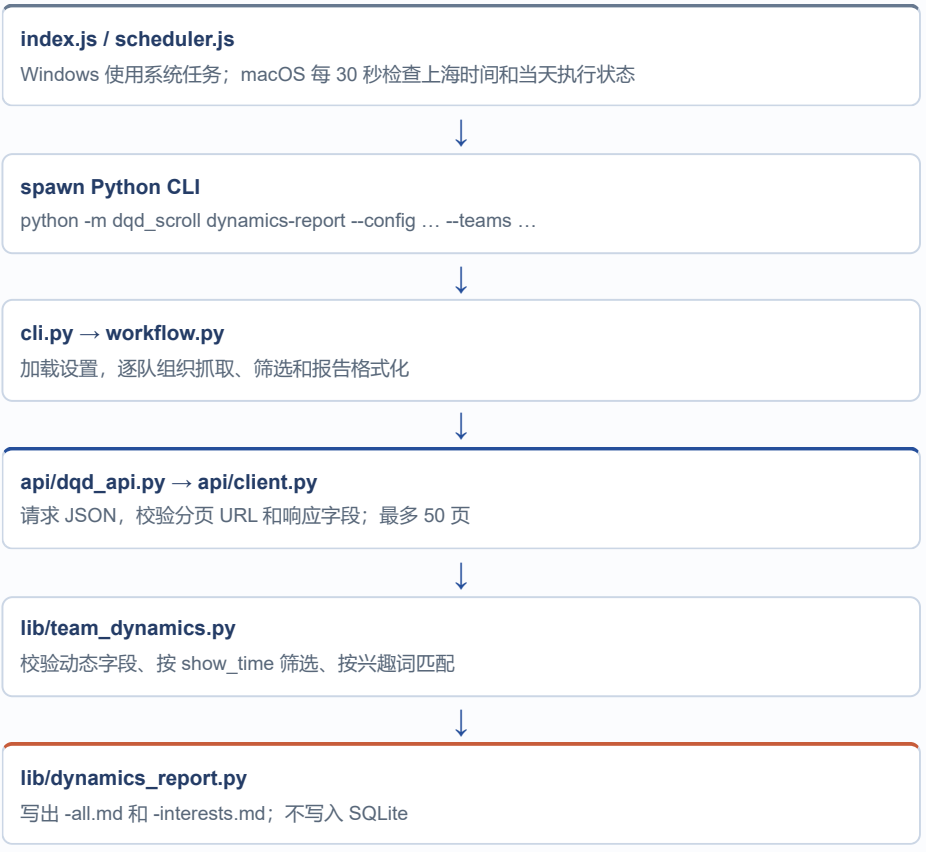
这条命令读取 SQLite，并把报告写到当前工作目录的 `out/<日期>.md`。它不访问网络，也不改写 SQLite。遇到数据库文件不存在时，程序会报错要求先初始化数据库，不会默默创建一个空数据库来掩盖路径问题。

路径会随命令入口不同而变化：`daily-report` 按当前工作目录写入 `out/<日期>.md`；`dynamics-report` 按 `--config` 文件所在目录写入 `out/dynamics/<日期>/`。插件把配置放在 DSH 用户数据目录，所以插件运行生成的动态报告也落在该用户目录下；从项目根目录手动运行时，结果则写进项目根目录的 `out/`。

6. 再看在线动态日报：一次命令经过哪些层

动态日报比赛程日报多了网络和分页，因此读起来更适合放在第二步。`config.toml` 的 `[dynamics]` 保存球队名称、兴趣词和运行时间；`teams.json` 将懂球帝球队名称映射到球队 ID。插件设置页修改用户配置，Python CLI 负责具体请求、筛选和报告内容。配置球队名称必须与映射表精确匹配；程序不会猜测 ID。

动态日报从定时器到两份 Markdown



关键实现边界如下：

- `api/client.py` 使用 Python 标准库 `urllib.request.urlopen` 发出 GET，检查 HTTP 状态并解析 JSON；当前代码没有引入 HTTPX。
- `api/dqd_api.py` 校验接口域名、路径、球队 ID 和分页 `type`，发现重复分页 URL 会停止；分页最多请求 50 页。
- `lib/team_dynamics.py` 把响应中的 `show_time` 转成上海本地时间，再按时间范围筛选；兴趣匹配是大小写不敏感的子串匹配。
- `workflow.py::build_team_dynamics_reports()` 把抓取、兴趣筛选和两种 Markdown 格式连接起来。它不调用 `Repository`，所以动态内容不会写进 SQLite。
- `index.js` 用固定的命令与参数数组启动 Python，不把任意 shell 命令交给动态内容；进程成功退出后才记录当天运行完成。

7. Python 文件各自负责什么

文件	负责的事	初学者读它时留意
`config.py`	读取 TOML 和球队 JSON，检查必需项	缺配置会在哪里报错?
`models.py`	定义球队、球员、赛程和动态对象	每个对象有哪些字段?
`api/client.py`	发 HTTP 请求并解码 JSON	HTTP 与 JSON 错误如何区分?
`api/dqd_api.py`	构造请求、读取分页并校验响应	哪些字段来自接口?
`lib/team_dynamics.py`	解析动态并筛选时间和兴趣词	解析与筛选为何分层?
`lib/team_roster.py` / `lib/team_schedule.py`	清洗样本并检查业务规则	缺字段会怎样?
`repository.py`	SQLite 建表、查询和事务写入	哪个方法会写数据库?
`workflow.py`	串联 API、仓库与报告格式化	它协调哪些模块?
`lib/report.py` / `lib/dynamics_report.py`	把结果写成 Markdown	格式与数据读取是否分离?
`cli.py`	用 `argparse` 解析参数并调用处理函数	新命令在哪里注册?

读函数先问输入、校验和输出是什么。`def` 定义函数，缩进表示代码块，`import` 引入模块，`return` 返回结果；CLI 的 `return 0` 表示成功退出，`raise ValueError(...)` 会带着具体原因停止。`parse_team_dynamic()` 用 `raw_dynamic.get('id')` 取 ID，再检查类型和值；不符合时明确报错，不会补默认 ID。

查看 CLI 帮助：`python -m dqd_scroll --help` 会列出子命令，`python -m dqd_scroll daily-report --help` 会列出参数。`python -m` 表示按包运行模块；本项目先执行 `src/dqd_scroll/__main__.py`，再进入 `cli.main()`。沿着实际函数调用逐步阅读，比从头看完整个文件更容易理清流程。

第三幕｜沿着现有结构增加一项功能

8. 先选一个不需要新接口的练习

适合第一次扩展的练习，是给已有赛程 Markdown 增加一个“已结束比赛数量”摘要。它只使用现有 SQLite 数据，不需要新 API，也不需要新增数据库表。开始编码前，先确认当前样本里用于识别完赛状态和比分的字段，再写清统计口径；如果这两个条件无法确认，就先停在字段核对。

这一小功能可以按如下顺序落地：

步骤	可能涉及的文件	只做这一层的工作
1. 定义结果	需求说明或 CLI 帮助文字	明确“已结束”的判定字段、纳入范围、输出示例
2. 读现有数据	`repository.py`、`models.py`	确认查询对象已经提供统计所需字段；避免重复查同一份数据库
3. 写计算	`lib/team_schedule.py` 或新的 `lib/team_summary.py`	输入赛程模型，输出统计结果；不发 HTTP、不写文件
4. 写展示	`lib/report.py`	把统计结果插入 Markdown；格式不负责重新查数据
5. 暴露命令	`cli.py`	增加明确的子命令和必填参数，复用仓库与 formatter
6. 再接 DSH	`index.js`、需要时的 `client.js` / `settings-api.js`	CLI 参数和输出稳定后，再决定是否增加按钮或定时调用

如果这个摘要只用于显示，不必为了它加一张表。只有新数据需要跨运行保留时，才考虑在 `repository.py` 中添加表或字段，并明确事务、唯一键和旧数据处理规则。这样能避免把“计算结果”误当成“必须存储的新状态”。

9. 如果功能要请求新的懂球帝接口

球队名单、赛程和球员数据不应照搬动态接口的 URL 或请求参数。项目的接口记录把这些请求列为待核验线索；即使一份响应样本能被解析，也不能单独证明 HTTP 方法、状态码、认证或分页契约。新接口按下面的次序接入：

增加一种在线数据的推荐调用链



编码顺序的原因是每层都能独立回答一个问题：请求拿到什么？解析器是否能识别？数据是否完整？需不需要保存？命令是否能被人单独运行？宿主是否真的需要调度它？如果先改设置页，往往会把未经验证的数据契约直接变成用户可见功能。

当前可复用的代码路径包括：

- 名单样本：`fixtures/team-roster.response.sample.json` → `lib/team_roster.py` → `workflow.save_team_roster_response()` → `Repository.replace_team_players()`。
- 赛程样本：`fixtures/team-schedule.response.sample.json` → `lib/team_schedule.py` → `workflow.save_team_schedule_response()` → `Repository.replace_team_schedule()`。
- 在线动态：`api/dqd_api.py` → `lib/team_dynamics.py` → `workflow.build_team_dynamics_reports()` → `lib/dynamics_report.py`。

10. 新功能的最小交付清单

检查项	开始写代码前应能回答
数据来源	来自 SQLite、经过核验的 HTTP 响应，还是用户配置？
样本依据	哪个样本证明字段路径、类型和可选性？

检查项	开始写代码前应能回答
失败行为	缺字段、类型变化、冲突记录或请求失败时，程序会在哪里停止？
存储需要	是否真的要跨运行保存？若要保存，唯一键与事务边界是什么？
入口	用户、定时器或设置页调用哪个固定 CLI 命令？必填参数是什么？
外部影响	这项功能会请求网络、写本地数据库，还是执行平台写入？
输出范围	报告写到哪里，是否会保存原始响应或敏感信息？

“每层只做自己的事”不是为了多建文件，而是为了让第一次失败时容易定位：请求错误看 `api/`，字段不符看 `lib/`，数据库错误看 `repository.py`，命令参数错误看 `cli.py`，宿主没启动进程才看 `index.js`。

工具箱｜后续迭代时保留哪些判断

11. 把 dsh-bili-scroll 当成结构参照，不当成代码模板

可以借鉴	应按懂球帝项目重新决定
插件 UI、任务编排和平台数据能力分层	B 站的 API、工具名、请求参数与响应字段
用户配置和运行状态放在用户数据目录	B 站 MCP server 与工具实现
限制可调用的入口，拒绝无边界的通用 shell	任何账号认证、Cookie 或设备标识的使用方式
日志与结果文件有固定位置和明确边界	内容摘要、用户互动与定时策略

懂球帝项目已经选定“JS 插件 + Python CLI”的交接方式。只有未来出现明确的 MCP 消费方或 MCP 工具契约需求时，才重新讨论协议；单纯增加一条日报、一次统计或一种数据解析，并不要求增加 MCP。

12. 当前边界与下一步优先级

优先级	工作	当前状态
1	先读懂 `daily-report` 的本地调用链	代码和本地赛程数据库路径已存在，不需要新接口
2	从脱敏样本改进名单/赛程解析或输出	离线导入和 SQLite 处理已实现
3	如需在线名单/赛程，重新核实真实请求契约	当前尚未实现 HTTP 请求；不要把旧 URL 当成已确认
4	核实球员资料、统计、能力值接口	现有 Postman 记录只是待复核线索
5	CLI 需求稳定后才让 DSH 调度	动态日报已有调度；新的赛程或球员功能尚未接入调度

现有动态接口的成功调用只能说明这一个接口在已记录的请求条件下曾成功，不能证明所有懂球帝接口都免认证、长期稳定或允许任意频率访问。若接口字段变化，解析器应报告不符合契约；若请求授权或平台规则不清楚，应先确认使用范围。请求只读也可能形成服务端访问记录，不应伪造浏览、点赞、评论或其他用户行为。

13. 证据范围

比较依据是本机两个项目在 2026-10-25 的工作区快照：dsh-bili-scroll v2.1.0 的 `package.json`、`index.js`、`client.js`、`lib/mcp.mjs`、`lib/state.mjs` 和 `vendor/bilibili-mcp/`；dsh-dqd-scroll v0.3.0 的源码、README、`package.json`、`pyproject.toml`、`environment.yml`、`tests/release-metadata.test.mjs` 与 `docs/api-contract.md`。这是代码结构与已记录接口证据的对照，不是对未来可用性或平台政策的保证。

懂球帝动态接口的 Postman 验证过程来自项目自身的契约记录；本指南没有独立重放请求。名单、赛程和球员接口仍列为待核验，不从样本响应或旧请求 URL 推断认证结论。运行环境准备方面，项目文档说明插件可在 DSH 用户数据目录准备 uv 和 Python 3.11；本报告没有在本机重新安装该环境。

参考资料

以下代码路径均相对于各自项目根目录。

dsh-bili-scroll

- `README.md`：插件运行方式、功能和本机数据边界。
- `index.js`、`client.js`：DSH 插件入口与设置页。
- `lib/mcp.mjs`：Node 与 MCP 子进程之间的 stdio 通信。
- `lib/state.mjs`：插件用户数据目录与状态。
- `prompts/duty.md`：一次任务的目标和边界。
- `vendor/bilibili-mcp/mcp_server.py`：内置 Python MCP 工具服务。
- `package.json`、`NOTICE.md`：插件元数据、依赖和第三方署名。

dsh-dqd-scroll

- `README.md`、`config.toml`、`teams.json`：项目用途、配置格式和球队映射。
- `index.js`、`client.js`、`settings-api.js`、`lib/user-data.js`：插件调度、设置页接口和本机用户数据。
- `environment.yml`、`pyproject.toml`、`src/dqd_scroll/__main__.py`、`src/dqd_scroll/__init__.py`：Python 环境、包元数据与命令入口。
- `src/dqd_scroll/cli.py`、`workflow.py`、`models.py`、`repository.py`：命令、流程编排、数据模型与 SQLite。
- `src/dqd_scroll/api/`、`src/dqd_scroll/lib/`：接口读取、响应处理、赛程加工与 Markdown 输出。
- `docs/api-contract.md`、`fixtures/`：接口证据记录和离线响应样本。
- `tests/release-metadata.test.mjs`：发布文件清单与 npm/Python 包版本一致性检查；当前 CLI 版本字符串不在该一致性断言中。

代码实现状态与接口证据分开描述。项目代码能证明“现在写了什么”；接口契约记录能证明“哪些请求曾被观察或重放”；两者都不能单独保证接口未来不变。